

Algoritmi-Modulo algoritmi per bioinformatica

Algoritmi avidi o golosi (greedy) - Capitolo 15 CLRS

Roberto Posenato

versione 2026-1, 17/05/2026

Sommario

- ① Introduzione
 - Idea generale
- ② Dettagli
 - Pseudocodifica
 - Considerazioni
- ③ Esempi di applicazione
 - Albero di ricoprimento minimo
 - Cammini minimi singola sorgente
- ④ Fondamenti teorici
 - Matroidi
 - Proprietà dei matroidi
 - Algoritmi greedy in un matroide pesato
 - Applicazione della teoria dei matroidi
- ⑤ Altri problemi risolvibili con greedy
- ⑥ Esercizi
 - Matroidi e minimizzazione
 - Programmazione attività di durata unitaria
 - Resto in monete

In questa lezione si richiamano i concetti fondamentali del paradigma *greedy*.

Il paradigma *greedy* prevede di risolvere un problema di **ottimizzazione** nel seguente modo:

- La soluzione è costruita in modo incrementale;
- A ogni iterazione, si aggiunge alla soluzione una componente scegliendo un elemento che risulta essere la scelta ottimale in quel momento.

Aspetti cruciali

- il problema viene risolto scegliendo iterativamente una componente della soluzione;
- la scelta della componente ottimale è di tipo locale;
- un algoritmo greedy determina una soluzione ottima quando il problema soddisfa 1) *la proprietà della scelta greedy* e 2) *la proprietà della sottostruttura ottima*.

Il paradigma *greedy* può essere così schematizzato:

Procedura GREEDY(X)

Input: $X = \{x_1, x_2, \dots, x_n\}$ insieme degli elementi candidati; criterio greedy di selezione ;
test di ammissibilità

Output: Un sottoinsieme $S = \{x_{i_1}, \dots, x_{i_k}\} \subseteq X$ che ottimizza il valore $f(x_{i_1}, \dots, x_{i_k})$.

- 1: $S = \emptyset$
 - 2: **Ordina** X secondo il criterio greedy di selezione
 - 3: **for** $i = 1$ **to** n **do**
 - 4: **if** (x_i *può essere aggiunto*) **then** $S = S \cup \{x_i\}$
 - 5: **return** S
-

- Deve essere possibile definire un criterio greedy che permetta di ordinare gli elementi in base alla loro “appetibilità” locale.
- La soluzione quindi è costruita in modo incrementale considerando gli oggetti uno alla volta e aggiungendo, ogni volta, quello più appetibile, se possibile.
- In altre parole, a ogni passo, si è in grado di determinare qual è l'elemento migliore da aggiungere senza considerare le scelte future possibili e senza dover considerare tutte le possibili relazioni esistenti.
- Un algoritmo *greedy* fornisce una soluzione ottima se si verifica che **il problema soddisfa la proprietà della scelta greedy e la proprietà della sottostruttura ottima.**

Esempi di applicazione

ALBERO DI RICOPRIMENTO MINIMO [Capitolo 21 CLRS]

PROBLEMA ALBERO DI RICOPRIMENTO MINIMO (MIN SPANNING TREE)

INPUT: Grafo $G = (V, E)$ non orientato, connesso di ordine n . Funzione *peso* $w: E \rightarrow \mathbb{R}$.

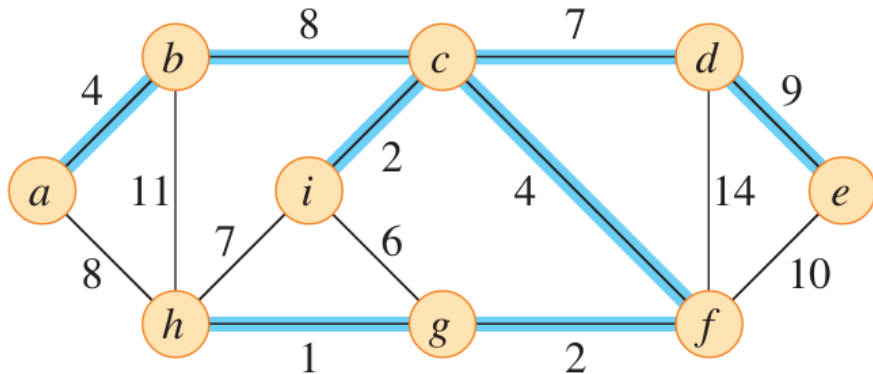
OUTPUT: Determinare l'albero di ricoprimento minimo $T = (V, E_T)$:

- albero di n nodi;
- $E_T \subseteq E$;
- peso $w(T) = \sum_{e \in E_T} w(e)$ è minimo.

L'insieme delle soluzioni ammissibili è l'insieme degli alberi di ordine n con archi in E . Qual è la sua cardinalità?

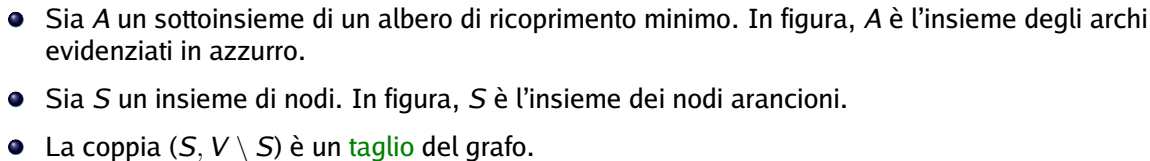
Esempi di applicazione

ALBERO DI RICOPRIMENTO MINIMO [Capitolo 21 CLRS]



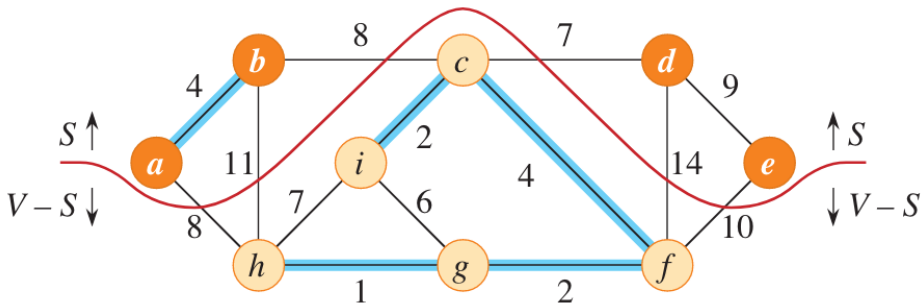
Albero di ricoprimento minimo per un grafo connesso. Il peso dell'albero è 37. L'albero non è l'unico minimo: togliendo l'arco (b, c) e aggiungendo l'arco (a, h) si ottiene un altro albero di ricoprimento minimo.

ALBERO DI RICOPRIMENTO MINIMO



Esempi di applicazione

ALBERO DI RICOPRIMENTO MINIMO



- Gli archi che **attraversano** il taglio sono gli archi con un nodo in S e l'altro in $V \setminus S$.
- Un taglio **rispetta** A se nessun arco di A attraversa il taglio.
- Un arco che attraversa un taglio avente il peso minimo tra tutti gli archi che attraversano il taglio è detto arco **leggero**.

Esempi di applicazione

ALBERO DI RICOPRIMENTO MINIMO

Teorema 1

Sia $G = (V, E)$ un grafo pesato e connesso; w la funzione peso; $A \subseteq E$ un insieme di archi che appartiene a un albero di ricoprimento minimo.

*Sia $(S, V \setminus S)$ un qualsiasi **taglio** di G che **rispetta** A e (u, v) un arco leggero del taglio. Allora (u, v) è un arco **sicuro** per A : si può aggiungere ad A e A rimane un sottoinsieme di un albero di ricoprimento minimo.*

Dimostrazione (sintesi dimostrazione 21.1 CLRS)

T = albero di ricoprimento minimo che contiene A .

Sia $(u, v) \notin T$ un arco leggero.

Si dimostra che si può costruire un altro albero di ricoprimento minimo T' che include A e (u, v) usando la tecnica **taglia-e-cuci** (**cut-and-paste**).

Esempi di applicazione

ALBERO DI RICOPRIMENTO MINIMO

Continua Dimostrazione (sintesi dimostrazione 21.1 CLRS)

- Poiché $(u, v) \notin T$, aggiungendolo a T si forma un ciclo p , dato che T è un albero di ricoprimento minimo.
- In T , u e v sono già connessi da un cammino semplice p' .
- Dato che u e v sono su lati opposti del taglio, almeno un altro arco $(x, y) \in p'$ attraversa il taglio.
- Anche $(x, y) \notin A$ perché il taglio rispetta A .
- Togliendo (x, y) da T , T si spezza in due parti separate.
- Aggiungendo (u, v) , le due parti si ricongiungono in un albero $T' = T - \{(x, y)\} + \{(u, v)\}$.
- $w(u, v) \leq w(x, y)$ perché (u, v) è un arco leggero.
- $w(T') \leq w(T)$, ma T è un albero di ricoprimento minimo per ipotesi: $w(T) \leq w(T')$.
- T' è quindi un albero di ricoprimento minimo.
- Dato che $A \subseteq T$ per ipotesi e $(x, y) \notin A$, vale $A \subseteq T'$.



Esempi di applicazione

ALBERO DI RICOPRIMENTO MINIMO

- L'algoritmo di Prim è un algoritmo greedy che risolve il problema ALBERO DI RICOPRIMENTO MINIMO.
- Implementa in modo efficiente il metodo indicato dal teorema 1.
- Si inizializza A , albero di ricoprimento minimo, con un nodo r del grafo.
- Finché esistono nodi non ancora raggiunti, si aggiunge ad A un arco leggero che connette un nodo non presente in A .
- Quando non ci sono più nodi da aggiungere, per il teorema 1, A è un albero di ricoprimento minimo.
- Per selezionare un nodo da unire, tutti i nodi non in A sono gestiti in una coda di min-priorità.
- La priorità di ciascun nodo v in coda è il peso minimo dei suoi archi incidenti a nodi in A .

Esempi di applicazione

ALBERO DI RICOPRIMENTO MINIMO

Algoritmo: PRIM

Input: $G = (V, E)$, funzione peso w , nodo di partenza $r \in V$.

Output: Un albero di ricoprimento minimo con radice r e definito tramite π .

```
1: foreach  $v \in V$  do
2:    $v.key := \infty$ 
3:    $v.\pi := NULL$ 
4:  $r.key := 0$ 
5:  $Q := V$ 
6: BUILDMINPRIORITYQUEUE( $Q$ )
7: while  $Q \neq \emptyset$  do
8:    $u := \text{EXTRACTMIN}(Q)$ 
9:   foreach  $v \in G.Adj[u]$  do
10:    if  $v \in Q$  and  $w(u, v) < v.key$  then
11:       $v.\pi := u$ 
12:      DECREASEKEY( $Q, v, w(u, v)$ )
```

// Q è una coda di min-priorità rispetto a key

// $v.key$ diventa $w(u, v)$

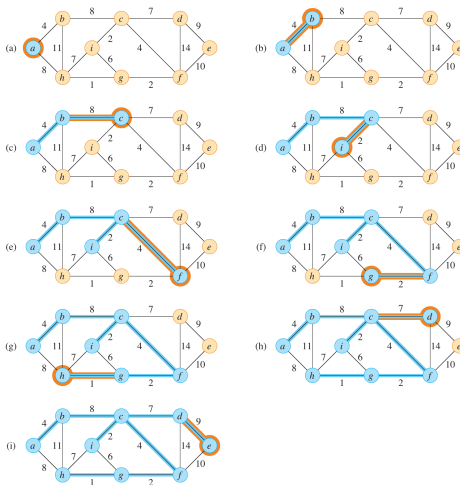
Esempi di applicazione

ALBERO DI RICOPRIMENTO MINIMO

- La fase di ordinamento è implicita nella coda di priorità; in coda si mettono i nodi rispetto al peso minimo degli archi che li collegano ai nodi che già compongono l'albero di ricoprimento minimo;
- Ciascun nodo è considerato solo una volta (a ogni ciclo **while**, quello con peso minimo);
- Nel ciclo **while**, dato un nodo u :
 - Tutti i suoi adiacenti vengono considerati;
 - Se un adiacente v **non** fa parte dell'albero (è ancora in Q) e il suo peso in coda è maggiore di $w(u, v)$, allora si aggiornano il collegamento tramite il suo arco leggero (assegnando $v.\pi$) e il suo peso in Q (DECREASEKEY).

Esempi di applicazione

ALBERO DI RICOPRIMENTO MINIMO



Esempi di applicazione

ALBERO DI RICOPRIMENTO MINIMO

L'analisi della complessità dipende da come si implementa la coda di min-priorità. **Caso con coda di priorità semplice e ricerca lineare del minimo:**

- $n = |V|, m = |E|$.
- Inizializzare i nodi: $O(n)$.
- Il ciclo **while** viene eseguito n volte.
- Ciascuna $\text{HEAPEXTRACTMIN}(Q)$: $O(n)$.
- Quindi lo svuotamento della coda: $O(n^2)$.
- Siccome il ciclo **for** analizza tutti gli adiacenti di ogni nodo, il totale di tutti i cicli è $2m$.
- L'appartenenza alla coda si può testare in tempo costante! (come?)
- Ciascun DECREASEKEY : $O(1)$.
- Il costo di tutti i cicli **for**: $O(m)$.

Costo finale: $O(n^2 + m) = O(n^2)$.

Esempi di applicazione

ALBERO DI RICOPRIMENTO MINIMO

Caso con coda di priorità implementata tramite min-heap:

- $n = |V|, m = |E|$.
- Inizializzare i nodi: $O(n)$.
- $\text{BUILDMINHEAP}(Q)$: $O(n)$.
- Il ciclo **while** viene eseguito n volte.
- Ciascuna $\text{HEAPEXTRACTMIN}(Q)$: $O(\lg n)$.
- Quindi lo svuotamento della coda: $O(n \lg n)$.
- Siccome il ciclo **for** analizza tutti gli adiacenti di ogni nodo, il totale di tutti i cicli è $2m$.
- L'appartenenza alla coda si può testare in tempo costante! (come?)
- Ciascun DECREASEKEY : $O(\lg n)$.
- Il costo di tutti i cicli **for**: $O(m \lg n)$.

Costo finale: $O(n + n \lg n + m \lg n) = O(m \lg n)$.

L'implementazione con heap conviene quando $m = o(n^2 / \lg n)$.

Esempi di applicazione

Cammini minimi da singola sorgente (SSSP, Single-Source Shortest Paths)

PROBLEMA CAMMINI MINIMI DA SINGOLA SORGENTE

INPUT: Grafo $G = (V, E)$ orientato e connesso di ordine n .
Funzione *peso* $w: E \rightarrow \mathbf{R}^+$ (solo valori non negativi!).
Un nodo $s \in V$.

OUTPUT: Determinare i **cammini minimi** da s verso tutti gli altri nodi.

Teorema 2 (Sottostruttura ottima)

Dato un cammino minimo $p = [v_1, v_2, \dots, v_k]$ dal vertice v_1 al vertice v_k , per qualsiasi i, j con $1 \leq i \leq j \leq k$, il sottocammino $p_{ij} = [v_i, v_{i+1}, \dots, v_j]$ è un cammino minimo da v_i a v_j .

Esempi di applicazione

Cammini minimi da singola sorgente (SSSP, Single-Source Shortest Paths)

Un algoritmo greedy è stato proposto da Dijkstra ['dɛik,stra] (1959) [Capitolo 22.3 CLRS]:

- Si inizializza una **stima del cammino minimo** d per ciascun nodo a $+\infty$.
- $s.d = 0$ per definizione.
- Si mantiene un insieme S di nodi per i quali si è già ottenuto un cammino minimo.
- A ogni passo, si considera un vertice $u \in V \setminus S$ con la **stima del cammino minimo minima**.
- Si aggiunge u a S e si **rilassano** (aggiornano) le stime del cammino minimo per tutti gli adiacenti di u .

Esempi di applicazione

Cammini minimi da singola sorgente (SSSP, Single-Source Shortest Paths)

Due sono gli aspetti da approfondire:

① Stima del cammino minimo:

- Per ogni nodo si deve mantenere un limite superiore alla lunghezza del cammino minimo dal nodo iniziale: $v.d$.
- Inizialmente $v.d = +\infty \quad \forall v \in V \setminus \{s\}, \quad s.d = 0$.

② Rilassamento:

- Quando si aggiunge un nodo u a un cammino minimo, si verifica se, passando per u , è possibile migliorare le stime dei cammini minimi verso i suoi adiacenti.
- Se $(u, v) \in E$ e $(v.d > u.d + w(u, v))$, allora $v.d := u.d + w(u, v)$.
- La scelta del nodo u da aggiungere si basa sulla sua distanza minima stimata, che diventa definitiva. Questa stima funziona **se e solo se** i pesi degli archi non sono negativi. L'algoritmo di Dijkstra funziona solo per grafi con pesi non negativi.

Esempi di applicazione

Cammini Minimi da Singola Sorgente: pseudocodifica

Algoritmo: DIJKSTRA

Input: $G = (V, E)$, $s \in V$, $w()$ (funzione peso)

Output: Cammini minimi da s rappr. come attributi d e π dei nodi

1: **foreach** $v \in V$ **do** $v.d := \infty$; $v.\pi := NULL$;

2: $s.d := 0$; $Q := V$; $S := \emptyset$;

3: BUILDMINPRIORITYQUEUE(Q)

// coda min-priorità rispetto a d

4: **while** ($S \subset V$) **do**

5: $u := \text{EXTRACTMIN}(Q)$

6: $S := S \cup \{u\}$

7: **foreach** $v \in G.\text{Adj}[u]$ **do**

8: **if** $v \in Q$ **and** $v.d > u.d + w(u, v)$ **then**

9: $v.d := u.d + w(u, v)$

10: DECREASEKEY($Q, v, v.d$)

11: $v.\pi := u$

Esempi di applicazione

Cammino minimo singola sorgente: analisi

Teorema 3 (Correttezza algoritmo di Dijkstra)

Dato un grafo $G = (V, E)$ con pesi non negativi w e sorgente s , l'algoritmo di Dijkstra termina e, per ogni $u \in V$, restituisce $u.d = \delta(s, u)$, distanza minima da s a u .

Dimostrazione

- Si consideri l'invariante di ciclo
“All'inizio di ogni ciclo **while**, $v.d = \delta(s, v)$ per ciascun nodo $v \in S$ ”.
- Basta mostrare che, per ogni $u \in V$, $u.d = \delta(s, u)$ nel ciclo in cui u è aggiunto a S .
- **Inizializzazione:** $S = \emptyset$. L'invariante è banalmente vero.

Esempi di applicazione

Cammino minimo singola sorgente: analisi

Dimostrazione (continua)

- **Mantenimento:** Si assume $v.d = \delta(s, v)$, $\forall v \in S$, e lo si dimostra anche per il nodo u aggiunto a S .
- Sia p cammino minimo da s a u .
- Sia y il primo nodo in p non in S e x il suo predecessore (che quindi è in S).
- y precede u o è $u \Rightarrow \delta(s, y) \leq \delta(s, u)$.
- Se u è stato selezionato, significa che $u.d \leq y.d$. Inoltre vale $\delta(s, u) \leq u.d$.
- Per ipotesi induttiva, $x.d = \delta(s, x)$ e quando x è stato aggiunto, $y.d$ è stato aggiornato a $\delta(s, y)$.
- Quindi: $\delta(s, y) \leq \delta(s, u) \leq u.d \leq y.d = \delta(s, y) \Rightarrow \delta(s, y) = \delta(s, u) = u.d = y.d$.

Esempi di applicazione

Cammino minimo singola sorgente: analisi

- $n = |V|, m = |E|$.
- Per il costo in tempo, l'analisi è simile a quella fatta per l'algoritmo di Prim.
- Se si implementa la coda di priorità come semplice lista: $O(n^2)$.
- Se si implementa la coda di priorità come min-heap: $O(m \lg n)$.
- Se si implementa la coda di priorità come Fibonacci heap: $O(m + n \lg n)$.

La teoria dei matroidi permette di stabilire se la struttura di un problema soddisfa le **condizioni sufficienti** affinché una soluzione greedy sia ottima.

Definizione 1 (Matroide)

Un **matroide** è una coppia ordinata $M = (S, \mathcal{I})$ che soddisfa le seguenti condizioni:

- 1 S è un insieme finito;
- 2 $\mathcal{I}$ è un insieme non vuoto di sottoinsiemi di S tali che $B \in \mathcal{I} \wedge A \subseteq B \Rightarrow A \in \mathcal{I}$; $\mathcal{I}$ è detta **ereditaria**. I sottoinsiemi sono detti **indipendenti** di S .
- 3 Proprietà **di scambio**: $A \in \mathcal{I}, B \in \mathcal{I} \wedge |A| < |B| \Rightarrow \exists x \in B \setminus A$ tale che $A \cup \{x\} \in \mathcal{I}$.

Esempio 1 (Matroide grafico)

Dato un grafo non orientato $G = (V, E)$, il matroide grafico $M_G = (S_G, \mathcal{I}_G)$ è definito da:

- $S_G = E$;
- Se $A \subseteq E$, allora $A \in \mathcal{I}_G$ se e solo se A è aciclico.
Ovvero, $\mathcal{I}_G$ contiene tutti i sottoinsiemi di archi che inducono una foresta.

Dimostrazione.

- 1 $S_G = E$ è un insieme finito perché consideriamo solo grafi finiti.
- 2 $\mathcal{I}_G$ è ereditario: ogni sottoinsieme di una foresta è una foresta, quindi appartiene a $\mathcal{I}$.

[continua...]



Dimostrazione.

3 Proprietà di scambio

- Si considerino due foreste $G_A = (V, A)$ e $G_B = (V, B)$ con $|A| < |B|$.
- Una foresta di $|V|$ nodi e con k archi contiene esattamente $|V| - k$ alberi.
- G_B ha meno alberi di $G_A \Rightarrow$ esiste un albero T di G_B che contiene vertici appartenenti a due alberi distinti di G_A .
- Si consideri allora un arco (u, v) di T tale che u e v appartengono a due alberi distinti di G_A .
- Se in G_A si aggiunge l'arco (u, v) , i due alberi di G_A contenenti u e v si fondono in un nuovo albero senza formare un ciclo (i nodi sono presenti una e una sola volta!).
- Quindi $A \cup \{(u, v)\} \in \mathcal{I}$.



Ancora poche definizioni...

- Un elemento $x \notin A$ è un'**estensione** di $A \in \mathcal{I}$ se può essere aggiunto preservando l'indipendenza di A .
- Un sottoinsieme $A \in \mathcal{I}$ è **massimale** se non ammette estensioni.
- Si dimostra che tutti i sottoinsiemi massimali di un matroide hanno la stessa dimensione (si applica banalmente la proprietà di scambio su un insieme massimale per ottenere un assurdo).
- Se un matroide ha una funzione peso $w: S \rightarrow \mathbb{R}^+$, allora è **pesato**. Il peso di un sottoinsieme A è $w(A) = \sum_{x \in A} w(x)$.

Matroide grafico (2)

Un matroide grafico ammette come sottoinsiemi massimali gli **alberi di ricoprimento**, ciascuno con $|V| - 1$ archi.

PROBLEMA MASSIMO DI UN MATROIDE PESATO

INPUT: Un matroide pesato $M = (S, \mathcal{I})$.

OUTPUT: Determinare un sottoinsieme di peso massimo $A \in \mathcal{I}$, detto anche **sottoinsieme ottimo**.

- Poiché $w()$ è positiva, il problema chiede un sottoinsieme **massimale di peso massimo**.
- Poiché ogni sottoinsieme di un massimale appartiene a $\mathcal{I}$, si può partire da $\emptyset$ e aggiungere l'elemento più *pesante* fino a ottenere un massimale (proprietà di scambio).
- Ma questo **è il paradigma greedy!**

Fondamenti teorici

Algoritmi greedy in un matroide pesato

Procedura GREEDY(M, w)

```
1:  $A = \emptyset$  // è un insieme indipendente  
2: Ordina  $S$  in ordine decrescente rispetto a  $w$   
3: for ( $i := 1$ ;  $i \leq n$ ;  $i++$ ) do  
4:   if ( $A \cup \{s_i\} \in \mathfrak{I}$ ) then  
5:      $A := A \cup \{s_i\}$   
6: return  $A$ 
```

- Invariante di ciclo: A è indipendente. Di seguito si dimostra che è massimale e ottimo.
- Il costo in tempo è dato dalla somma:
 - costo di ordinamento: $O(n \log n)$ dove $n = |S|$.
 - $n \times$ costo verifica($A \cup \{s_i\} \in \mathfrak{I}$), che si assume pari a $f(n) \Rightarrow O(nf(n))$.

Totale $O(n \log n + nf(n))$.

Fondamenti teorici

Algoritmi greedy in un matroide pesato

Lemma 1 (Unicità dell'estensione)

Se un elemento $x \in S$ non è un'estensione di $\emptyset$, allora x non è un'estensione di qualsiasi sottoinsieme indipendente A di S .

Dimostrazione.

È sufficiente dimostrare la contronominale:

Se x è un'estensione di qualche sottoinsieme indipendente A , allora x è estensione di $\emptyset$.

Lasciata per esercizio. □

Quindi, se un elemento non può essere aggiunto quando viene considerato, non potrà mai essere aggiunto (né prima, né dopo).

Fondamenti teorici

Algoritmi greedy in un matroide pesato

Lemma 2 (Elemento massimo in una soluzione ottima)

Se S è ordinato in modo decrescente e x è il primo elemento con $\{x\}$ indipendente (se esiste), allora esiste un sottoinsieme ottimo A che lo contiene.

Dimostrazione.

- Sia B un sottoinsieme ottimo tale che $x \notin B$.
- Si costruisca A da $A = \{x\}$ usando la proprietà di scambio con B .
- Quando $|A| = |B|$, allora $A = B \setminus \{y\} \cup \{x\}$ per un $y \in B$. Ma $w(y) \leq w(x)$ per ipotesi. Quindi $w(A) \geq w(B)$.
Quindi anche A è ottimo.



Fondamenti teorici

Algoritmi greedy in un matroide pesato

Teorema 4 (Proprietà della sottostruttura ottima dei matroidi)

Sia x il primo elemento di S scelto dalla procedura greedy con input $M = (S, \mathcal{I})$. Il problema di trovare un sottoinsieme di peso massimo che contiene x si riduce a trovare un sottoinsieme di peso massimo per il sottomatroide $M' = (S', \mathcal{I}')$.

$$S' = \{y \in S \mid \{x, y\} \in \mathcal{I}\}$$

$$\mathcal{I}' = \{B \subseteq S' \mid B \cup \{x\} \in \mathcal{I}\}$$

e la funzione peso è adeguata di conseguenza.

Dimostrazione.

- Se A è sottoinsieme indipendente per M , allora $A' = A \setminus \{x\}$ è sottoinsieme indipendente per M' e viceversa.
- Vale $w(A) = w(A') + w(x)$. Se A o A' ha peso massimo, anche l'altro insieme deve avere peso massimo nel proprio matroide.



Teorema 5 (Correttezza dell'algoritmo greedy con i matroidi)

Sia $M = (S, \mathcal{I})$ un matroide pesato con funzione peso w ; allora $\text{greedy}(M, w)$ determina un sottoinsieme ottimo.

Dimostrazione.

- Per il lemma 1, l'algoritmo non pregiudica la determinazione della soluzione considerando gli elementi una sola volta.
- Per il lemma 2, può aggiungere l'elemento massimo: esiste una soluzione ottima che lo contiene.
- Il ciclo **for** applica il Teorema 4: aggiunto l'elemento più grande, si risolve il problema sul sottomatroide M' .



PROBLEMA PROGRAMMAZIONE ATTIVITÀ DI DURATA UNITARIA SU SINGOLO PROCESSORE (UNIT-TIME TASK SCHEDULING)

INPUT: $S = \{a_1, a_2, \dots, a_n\}$ attività di durata unitaria. $D = \{d_1, \dots, d_n \mid 1 \leq d_i \leq n\}$ scadenze delle attività. $W = \{w_1, \dots, w_n \mid w_i \geq 0\}$ penalità da pagare. Si paga un costo w_i se l'attività a_i non termina entro il tempo d_i .

OUTPUT: Determinare un piano di programmazione per singolo processore che minimizza le penalità totali.

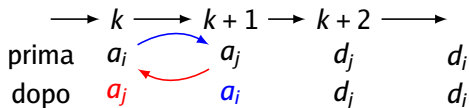
Terminologia e osservazioni:

- Un'attività è in **ritardo** se termina dopo la scadenza, in **anticipo** altrimenti.
- Un piano generico può essere sempre riordinato elencando prima le attività in anticipo e poi quelle in ritardo.

Fondamenti teorici

Applicazione della teoria dei matroidi

- Se le attività in anticipo sono ordinate per scadenza crescente, la programmazione è in **forma canonica**.
 - Si considerino due attività in anticipo a_i e a_j che terminano agli istanti k e $k + 1$.
 - a_j in anticipo $\Rightarrow k + 1 \leq d_j$.
 - Se $d_j < d_i$, allora $k + 1 < d_i$ e quindi scambiando a_i con a_j le due attività rimangono ancora in anticipo.



Fondamenti teorici

Applicazione della teoria dei matroidi

- Il problema consiste nel determinare un opportuno sottoinsieme ottimo delle attività in anticipo!
- Una volta trovato, si elencano prima le attività in anticipo e poi quelle in ritardo.
- Un insieme di attività è indipendente se nessuna è in ritardo.
- Sia $\mathcal{I}$ l'insieme di tali sottoinsiemi.
- Sia $N_t(A)$, per $t = 1, \dots, n$, il numero di attività di A con scadenza $\leq t$.
- Si dimostra che A è indipendente se e solo se $N_t(A) \leq t$, per $t = 1, \dots, n$.

Fondamenti teorici

Applicazione della teoria dei matroidi

Il problema di minimizzare la somma delle penalità delle attività in ritardo è equivalente al problema di massimizzare la somma delle penalità delle attività in anticipo.

Teorema 6

Sia S l'insieme delle attività unitarie con penalità e $\mathcal{I}$ l'insieme di tutti gli insiemi indipendenti di attività. Allora $M = (S, \mathcal{I})$ è un matroide.

Dimostrazione.

- Dimostriamo solo la proprietà di scambio, lasciando la dimostrazione delle altre due proprietà per esercizio.
- Siano B e A due sottoinsiemi indipendenti con $|B| > |A|$. Sia k il più grande t tale che $N_t(B) \leq N_t(A)$.
- Poiché $N_n(B) = |B|$, $N_n(A) = |A|$ e $|B| > |A|$, vale $k < n$ e $N_j(B) > N_j(A)$ per tutti i $k + 1 \leq j \leq n$.
Quindi $B \setminus A$ contiene almeno un'attività con scadenza $k + 1$.
- Sia a_i un'attività in $B \setminus A$ con scadenza $k + 1$ e sia $A' = A \cup \{a_i\}$. Per $k < t \leq n$ si ha che $N_t(A') \leq N_t(B) \leq t$ dato che B è indipendente. Dunque A' è indipendente.



- Si può quindi applicare il paradigma greedy per risolvere il problema.
- Il costo in tempo è $O(n^2)$:
 - $O(n \lg n)$ per ordinare le attività in modo decrescente rispetto alla loro penalità.
 - n cicli, in ciascuno dei quali si decide se aggiungere un'attività a_i alla soluzione.
 - Al ciclo j , si aggiunge a_j ad A se $A \cup \{a_j\}$ resta indipendente, cioè se $N_j(A \cup \{a_j\}) \leq j$.
 - La verifica di indipendenza costa $O(|A|)$.

Fondamenti teorici

Applicazione della teoria dei matroidi

Esempio 2

Le attività sono ordinate in modo decrescente rispetto alla loro penalità:

a_i	1	2	3	4	5	6	7
d_i	4	2	4	3	1	4	6
w_i	70	60	50	40	30	20	10

- Si mantiene un array s di n elementi inizializzato a '-'.
 - A partire da a_1 , si analizza un'attività alla volta.
 - Si colloca a_i nella prima posizione libera di s , partendo da d_i e scendendo a 1. (Caso peggiore: $O(d_i)$).
 - Se non c'è posizione libera, l'attività si scarta.
 - Finito il ciclo, $s = [a_4, a_2, a_3, a_1, -, a_7, -]$.
 - Si ordinano le attività in s rispetto alla scadenza.
 - Si aggiungono le due attività rimaste (con penalità) per ottenere la soluzione canonica.

- I problemi sui grafi visti precedentemente sono problemi su matroidi grafici pesati.
- Esistono però anche problemi la cui struttura non è rappresentabile da un matroide ma che ammettono soluzioni greedy.
- L'esempio più interessante di tali problemi è il *problema della compressione di file*, risolvibile in tempo polinomiale con l'algoritmo di Huffman.

Altri problemi risolvibili con greedy

Elenco di problemi importanti che ammettono un buon algoritmo greedy di risoluzione:

Problema	Costo naïve	Costo greedy
Albero minimo di ricoprimento		Algoritmo di Prim: $O(n^2)$
Schedulazione su macchina singola con minimizzazione del numero di attività in ritardo	$O(n^2)$	Algoritmo di Moore–Hodgson: $O(n \log n)$
Compressione di file		Codice di Huffman: $O(n \log n)$
Soddisfacibilità delle clausole di Horn (disgiunzioni con al più un letterale positivo)	$O(2^n)$	$O(n)$

Esercizi

Cammini minimi da sorgente singola

Esercizio 1

Completare l'algoritmo di Dijkstra in modo che restituisca anche i cammini minimi, oltre ai valori minimi.

Esercizio 2

Spiegate come dovrebbe essere cambiata la funzione peso di un matroide se si vuole trovare una soluzione di peso minimo tra gli insiemi massimali.

Esercizio 3

Dimostrare che un insieme A è indipendente se e solo se $N_t(A) \leq t$ e che è possibile verificare l'indipendenza in tempo $O(|A|)$.

Esercizio 4

Il problema del resto in monete consiste nel formare il resto di n ¢ usando il minor numero possibile di monete. Si assume che il valore delle monete sia un intero.

- 1 Se le monete sono da 1, 5, 10 e 20¢, dimostrare che esiste un algoritmo greedy che determina la soluzione ottima.
- 2 Lo stesso se le monete sono $c^0, c^1, \dots, c^k$ ¢ con $c > 1$ e $k \geq 1$.
- 3 Definite un insieme di monete per il quale l'algoritmo greedy può non determinare la soluzione ottima. L'insieme deve includere la moneta da 1¢ per garantire l'esistenza della soluzione.